HW2

Alex Villagomez

2026-01-30

Contents

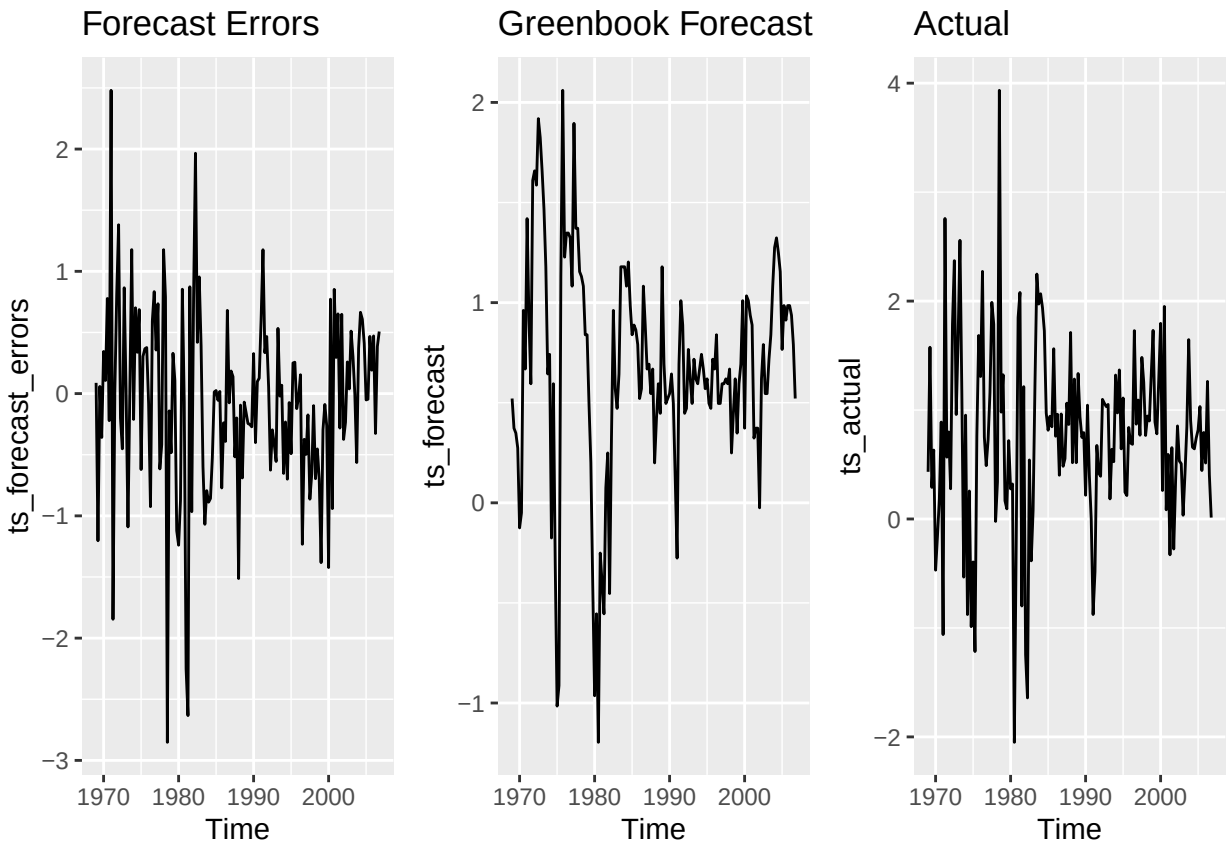
Problem 1

```
p1_data<-read_excel("../../data/Chapter4_exercises_data.xls", sheet="Exercise 6")
# The 1971:Q1 Greenbook forecast cell in the data file holds a stray "`" instead of a number,
# which makes read_excel read the column as text. Restore the value (5.8% annualized growth,
# converted to a quarterly rate like the rest of the column) and convert back to numeric.
p1_data[[3]][p1_data$DATE == "1971:Q1"] <- ((1 + 5.8/100)^(1/4) - 1) * 100
p1_data[[3]] <- as.numeric(p1_data[[3]])
head(p1_data)
```

```
## # A tibble: 6 x 3
##   DATE    `Actual RGDP Quarterly Growth (in %)` Greenbook RGDP Quarterly Growt~1
##   <chr>                                   <dbl>                            <dbl>
## 1 1969:Q1                                 0.433                            0.521
## 2 1969:Q2                                 1.58                             0.373
## 3 1969:Q3                                 0.290                            0.348
## 4 1969:Q4                                 0.633                            0.274
## 5 1970:Q1                                -0.470                           -0.125
## 6 1970:Q2                                -0.157                           -0.0500
## # i abbreviated name: 1: `Greenbook RGDP Quarterly Growth Forecast (in %)`
```

```
p1_data$forecast_errors <- p1_data[, 3] - p1_data[, 2]
ts_forecast<-ts(p1_data[,3], start=c(1969, 1), frequency=4)
ts_actual<-ts(p1_data[,2], start=c(1969, 1), frequency=4)
ts_forecast_errors<-ts(p1_data$forecast_errors, start=c(1969, 1), frequency=4)


plot1=autoplot(ts_forecast_errors, main="Forecast Errors")
plot2=autoplot(ts_forecast, main="Greenbook Forecast")
plot3=autoplot(ts_actual, main="Actual")
grid.arrange(plot1, plot2, plot3, ncol=3)
```



All three of these time series seem to be stationary. The y limits on the forecast errors are the lowest out of the three, followed by forecast, and last the actual values. The volatility is highest in earlier years but goes down over time.

```
summary(ts_forecast_errors)
```

```
##  Greenbook RGDP Quarterly Growth Forecast (in %)
##  Min.   :-2.85204
##  1st Qu.:-0.48582
##  Median :-0.05326
##  Mean   :-0.07746
##  3rd Qu.: 0.37236
##  Max.   : 2.48030
```

```
sd(ts_forecast_errors)
```

```
## [1] 0.7486704
```

```
summary(ts_forecast)
```

```
##  Greenbook RGDP Quarterly Growth Forecast (in %)
##  Min.   :-1.1963
##  1st Qu.: 0.4901
##  Median : 0.6683
```

```
##  Mean   : 0.6765
##  3rd Qu.: 0.9914
##  Max.   : 2.0604
```

```
sd(ts_forecast)
```

```
## [1] 0.553939
```

```
summary(ts_actual)
```

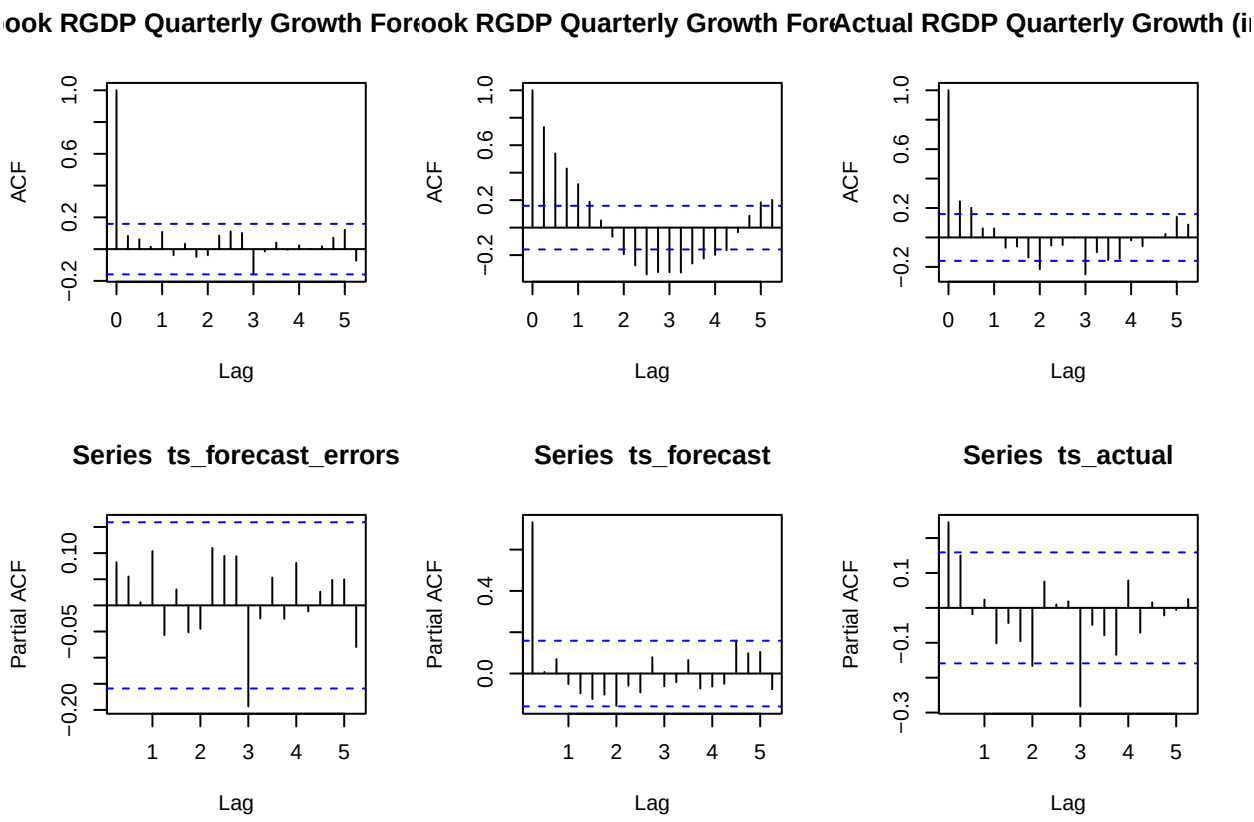
```
##  Actual RGDP Quarterly Growth (in %)
##  Min.   :-2.0497
##  1st Qu.: 0.3475
##  Median : 0.7679
##  Mean   : 0.7539
##  3rd Qu.: 1.1741
##  Max.   : 3.9343
```

```
sd(ts_actual)
```

```
## [1] 0.8423882
```

The forecast is off by a mean of -.077 which is quite close to 0 meaning that there is not much bias in the forecasts for either underestimating or overestimating, however, the standard deviation of .748 shows that the spread of the data means that the forecasts are usually off by around .75 percentage points. The means for the forecast and actual values are roughly similar, with a smaller mean for the forecasts which have a mean expected quarterly growth of .68% for GDP while the actual mean growth is .75%. This is an underestimate overall, which is reflected in the mean of forecast errors. The standard deviation for the forecast is lower than the standard deviation for the actual GDP growth, indicating that the forecast is more conservative in its estimates compared to the actual variance of the data.

```
par(mfrow=c(2,3))
acf(ts_forecast_errors)
acf(ts_forecast)
acf(ts_actual)
pacf(ts_forecast_errors)
pacf(ts_forecast)
pacf(ts_actual)
```



Yes, I do observe time dependence in ACF and PACF of the data. It is most prevalent in the forecast where the ACF has a clear pattern which indicates some sort of cycle component in the data. The actual GDP growth also has significant autocorrelation at the first lag or two. For the forecast errors, there is no autocorrelation as the spikes do not surpass the significance bands.

Problem 2

The mean of the forecast errors is slightly negative (-.077), which suggests that the forecast underestimates GDP growth on average. However, the intercept in the regression below is not statistically significant (p-value of .306), so we cannot reject that the expected value of the forecast errors is 0.

```
model <- dynlm(ts_forecast_errors ~ L(ts_forecast_errors, 1) + L(ts_forecast_errors, 2) +  L(ts_forecas
summary(model)
```

```
##
## Time series regression with "ts" data:
## Start = 1971(1), End = 2006(4)
##
## Call:
## dynlm(formula = ts_forecast_errors ~ L(ts_forecast_errors, 1) +
##     L(ts_forecast_errors, 2) + L(ts_forecast_errors, 3) + L(ts_forecast_errors,
##     4) + L(ts_forecast_errors, 5) + L(ts_forecast_errors, 6) +
##     L(ts_forecast_errors, 7) + L(ts_forecast_errors, 8))
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.73142 -0.36649  0.04607  0.44197  2.42503
```

```
##
## Coefficients:
##                           Estimate Std. Error t value Pr(>|t|)
## (Intercept)               -0.06835    0.06649  -1.028    0.306
## L(ts_forecast_errors, 1)  0.08947    0.08613   1.039    0.301
## L(ts_forecast_errors, 2)  0.03624    0.08613   0.421    0.675
## L(ts_forecast_errors, 3)  0.01226    0.08617   0.142    0.887
## L(ts_forecast_errors, 4)  0.11314    0.08608   1.314    0.191
## L(ts_forecast_errors, 5) -0.04816    0.08607  -0.560    0.577
## L(ts_forecast_errors, 6)  0.03450    0.08628   0.400    0.690
## L(ts_forecast_errors, 7) -0.04950    0.08562  -0.578    0.564
## L(ts_forecast_errors, 8) -0.04427    0.08542  -0.518    0.605
##
## Residual standard error: 0.7691 on 135 degrees of freedom
## Multiple R-squared:  0.02943,    Adjusted R-squared:  -0.02809
## F-statistic: 0.5117 on 8 and 135 DF,  p-value: 0.846
```

According to the regression results from the significance of the parameters, none of the lags of the forecast error are significant to the forecast error in time t. The F-test agrees that the past forecast errors are not able to be used to predict the forecast error.

Problem 3

```r
# MA(2): theta1 = -2, theta2 = 1.35, sigma^2 = 1
theta1 <- -2
theta2 <- 1.35
sig2 <- 1

# Autocovariances
g0 <- sig2 * (1 + theta1^2 + theta2^2)
g1 <- sig2 * (theta1 + theta1 * theta2)
g2 <- sig2 * theta2

# Theoretical ACF: rho(k) = gamma(k)/gamma(0)
rho0 <- 1
rho1 <- g1 / g0
rho2 <- g2 / g0
rho_k <- c(rho0, rho1, rho2, rep(0, 8))   # rho(0),...,rho(10)

# Display lags 0-10
theoretical_acf <- data.frame(lag = 0:10, rho = rho_k)
print(theoretical_acf)
```

```
##    lag        rho
## 1    0  1.0000000
## 2    1 -0.6888970
## 3    2  0.1978747
## 4    3  0.0000000
## 5    4  0.0000000
## 6    5  0.0000000
## 7    6  0.0000000
## 8    7  0.0000000
## 9    8  0.0000000
## 10   9  0.0000000
```

```
## 11  10  0.0000000
```

b)

```
set.seed(234)
n <- 100
e <- rnorm(n, mean = 0, sd = 1)   # white noise

# MA(2): y_t = 0.7 - 2*e_{t-1} + 1.35*e_{t-2} + e_t
y <- numeric(n)
y[1] <- 0.7 + e[1]                    # no lagged e
y[2] <- 0.7 - 2*e[1] + e[2]           # one lag
for (t in 3:n) {
  y[t] <- 0.7 - 2*e[t-1] + 1.35*e[t-2] + e[t]
}

# Sample ACF up to lag 10
sample_acf <- acf(y, lag.max = 10, plot = FALSE)
sample_acf_df <- data.frame(lag = 0:10, acf = c(1, sample_acf$acf[-1]))
print(sample_acf_df)
```

```
##    lag         acf
## 1    0  1.00000000
## 2    1 -0.58349312
## 3    2 -0.03302010
## 4    3  0.15145552
## 5    4  0.06837913
## 6    5 -0.18524788
## 7    6  0.05482998
## 8    7  0.15398866
## 9    8 -0.14060532
## 10   9 -0.08638002
## 11  10  0.17231815
```

Lag 1 lines up somewhat closely with the theoretical value (-.58 compared to -.69), but lag 2 does not (-.03 compared to .20). Later lags should theoretically go to 0, but that does not happen here as they stay between roughly -.19 and .17.

Problem 4

a)

```
# Estimate MA(2) on the simulated series (use the same 'y' from the simulation chunk)
fit_ma2 <- arima(y, order = c(0, 0, 2), include.mean = TRUE)
fit_ma2
```

```
##
## Call:
## arima(x = y, order = c(0, 0, 2), include.mean = TRUE)
##
## Coefficients:
##          ma1     ma2  intercept
```

```
##       -1.5589  0.7606     0.6939
## s.e.   0.0686  0.0735     0.0242
##
## sigma^2 estimated as 1.43:  log likelihood = -161.39,  aic = 330.79
```

The estimates do not exactly match up so there is a difference with the theoretical model. For this artificial MA(2), ma1 was -1.56 compared to -2 and ma2 was .76 compared to 1.35, but the intercept stayed roughly the same compared to before.

b)

```
# Given: e_t = 0.4, e_{t-1} = -1.2; theoretical mu=0.7, theta1=-2, theta2=1.35
mu <- 0.7
theta1 <- -2
theta2 <- 1.35
et <- 0.4
et_lag1 <- -1.2

# 1-step: E(y_{t+1}|I_t) = mu + theta1*e_t + theta2*e_{t-1}
f1 <- mu + theta1 * et + theta2 * et_lag1
# 2-step: E(y_{t+2}|I_t) = mu + theta2*e_t
f2 <- mu + theta2 * et
# 3-step: E(y_{t+3}|I_t) = mu
f3 <- mu

data.frame(horizon = 1:3, forecast = c(f1, f2, f3))
```

```
##   horizon forecast
## 1       1    -1.72
## 2       2     1.24
## 3       3     0.70
```

The forecasts for 1, 2, and 3 steps ahead are -1.72, 1.24, and 0.7.

Problem 5

For the first part, the process is invertible and we can get AR(inf)=1.2+.8(y(t-1)-1.2)-.8^2(y(t-2)-1.2)... for infinite iterations. The second is not invertible as since abs(theta) is more than 1, the series will diverge in the AR process. Here, it is preferred to use the MA process because it will not diverge for both and it uses less parameters.

Problem 6

```
# Download GOOG from Yahoo Finance (end date pinned to the submission date)
getSymbols("GOOG", src = "yahoo", from = "2020-01-01", to = "2026-01-30")
```

```
## [1] "GOOG"
```

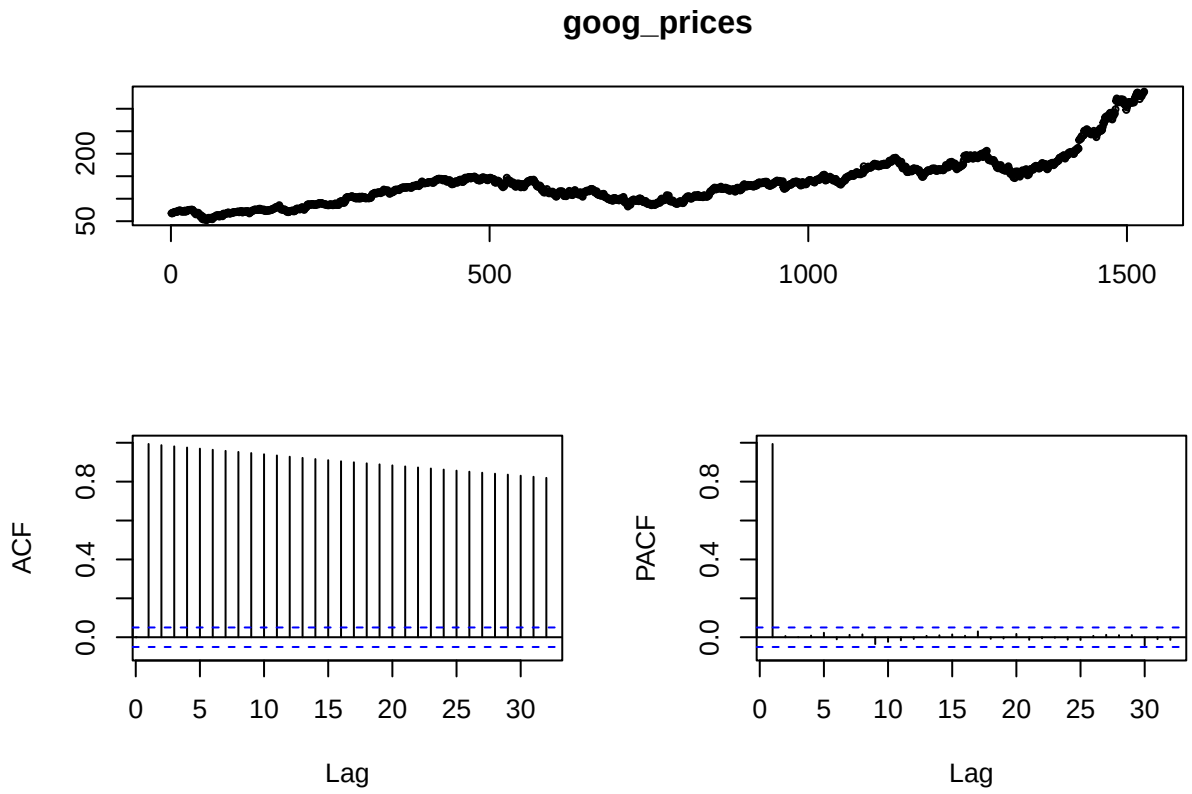
```
head(GOOG)
```

```
##            GOOG.Open GOOG.High GOOG.Low GOOG.Close GOOG.Volume GOOG.Adjusted
## 2020-01-02   67.0775   68.4070  67.0775    68.3685    28132000      67.72701
## 2020-01-03   67.3930   68.6250  67.2772    68.0330    23728000      67.39466
```

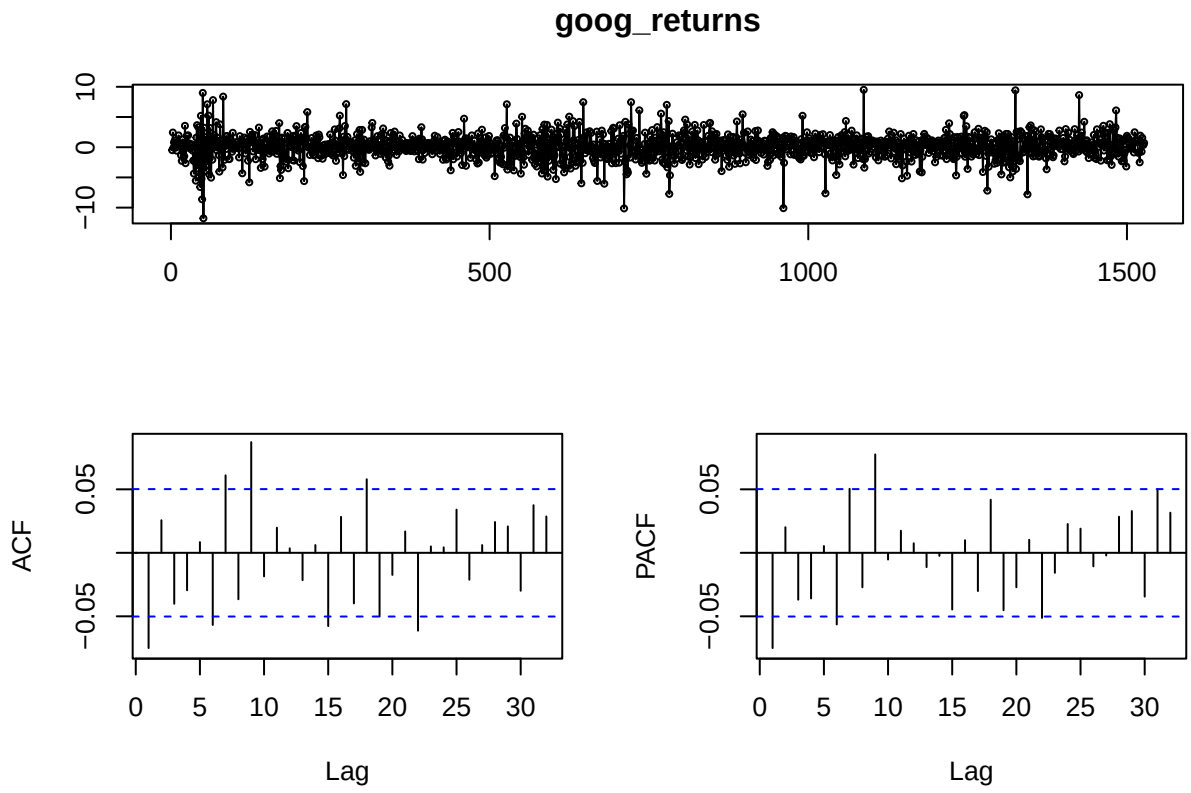
```
## 2020-01-06   67.5000   69.8250   67.5000    69.7105    34646000       69.05643
## 2020-01-07   69.8970   70.1495   69.5190    69.6670    30054000       69.01334
## 2020-01-08   69.6040   70.5790   69.5420    70.2160    30560000       69.55718
## 2020-01-09   71.0285   71.3665   70.5135    70.9915    30018000       70.32540
```

```
# Extract adjusted closing prices
goog_prices <- GOOG$GOOG.Adjusted
goog_returns <- ts(diff(log(goog_prices)) * 100)

tsdisplay(goog_prices)
```




```
tsdisplay(goog_returns)
```




```
model_ar1 <- arima(goog_returns, order = c(1, 0, 0), include.mean = TRUE)
summary(model_ar1)
```

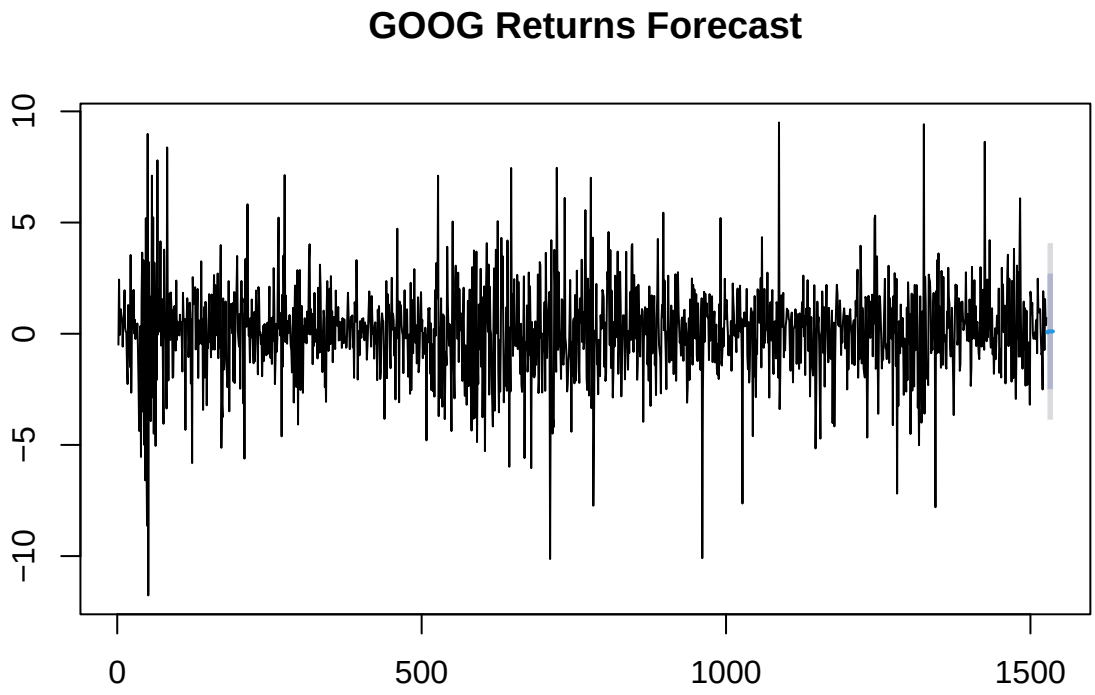
```
##
## Call:
## arima(x = goog_returns, order = c(1, 0, 0), include.mean = TRUE)
##
## Coefficients:
##           ar1  intercept
##       -0.0750     0.1053
## s.e.   0.0255     0.0481
##
## sigma^2 estimated as 4.081:  log likelihood = -3238.31,  aic = 6482.62
##
## Training set error measures:
##                         ME     RMSE   MAE     MPE     MAPE      MASE
## Training set -2.835462e-05 2.020109 1.446 92.3747 123.5938 0.6775138
##                      ACF1
## Training set 0.001465943
```

```
forecasts <- predict(model_ar1, n.ahead = 10)
data.frame(
  horizon = 1:10,
  forecast = round(forecasts$pred, 4),
```

```
  se = round(forecasts$se, 4)
)
```

```
##    horizon forecast     se
## 1        1   0.0604 2.0201
## 2        2   0.1087 2.0258
## 3        3   0.1051 2.0258
## 4        4   0.1054 2.0258
## 5        5   0.1053 2.0258
## 6        6   0.1053 2.0258
## 7        7   0.1053 2.0258
## 8        8   0.1053 2.0258
## 9        9   0.1053 2.0258
## 10      10   0.1053 2.0258
```

```
# Plot forecast
plot(forecast(model_ar1, h = 10), main = "GOOG Returns Forecast")
```



Based on the GOOG prices ACF and PACF, we should use an AR(1) model as the ACF slowly declines and the PACF cuts off after the first lag. After differencing the data to get returns, the ACF and PACF is consistent with white noise. I fit the AR(1) to GOOG returns and forecast 10 days ahead. The forecast is consistent with the data, as it quickly reverts to the mean daily return of about .1% with wide prediction intervals.

Problem 7

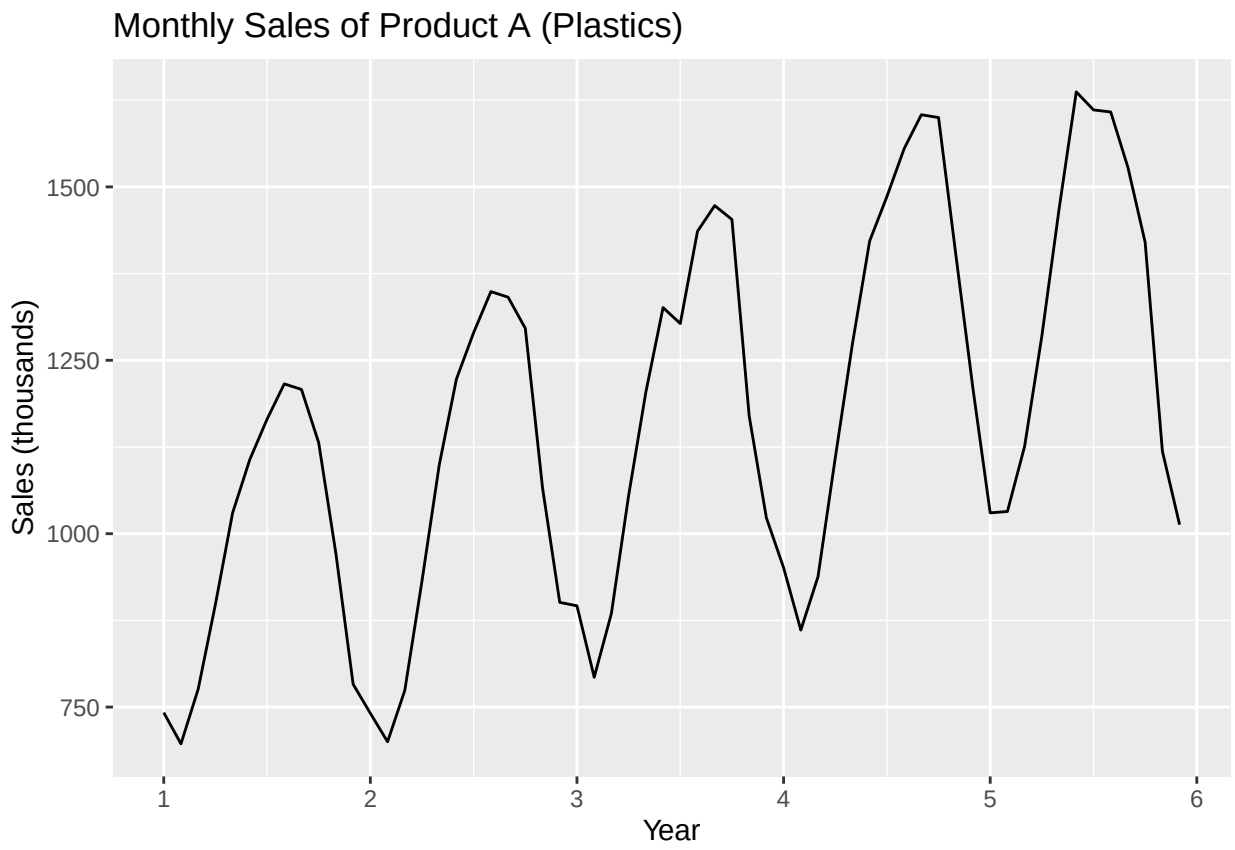
a)

```
library(fpp2)  # contains plastics dataset

# Load data
data(plastics)
head(plastics)
```

```
##     Jan  Feb  Mar  Apr  May  Jun
## 1   742  697  776  898 1030 1107
```

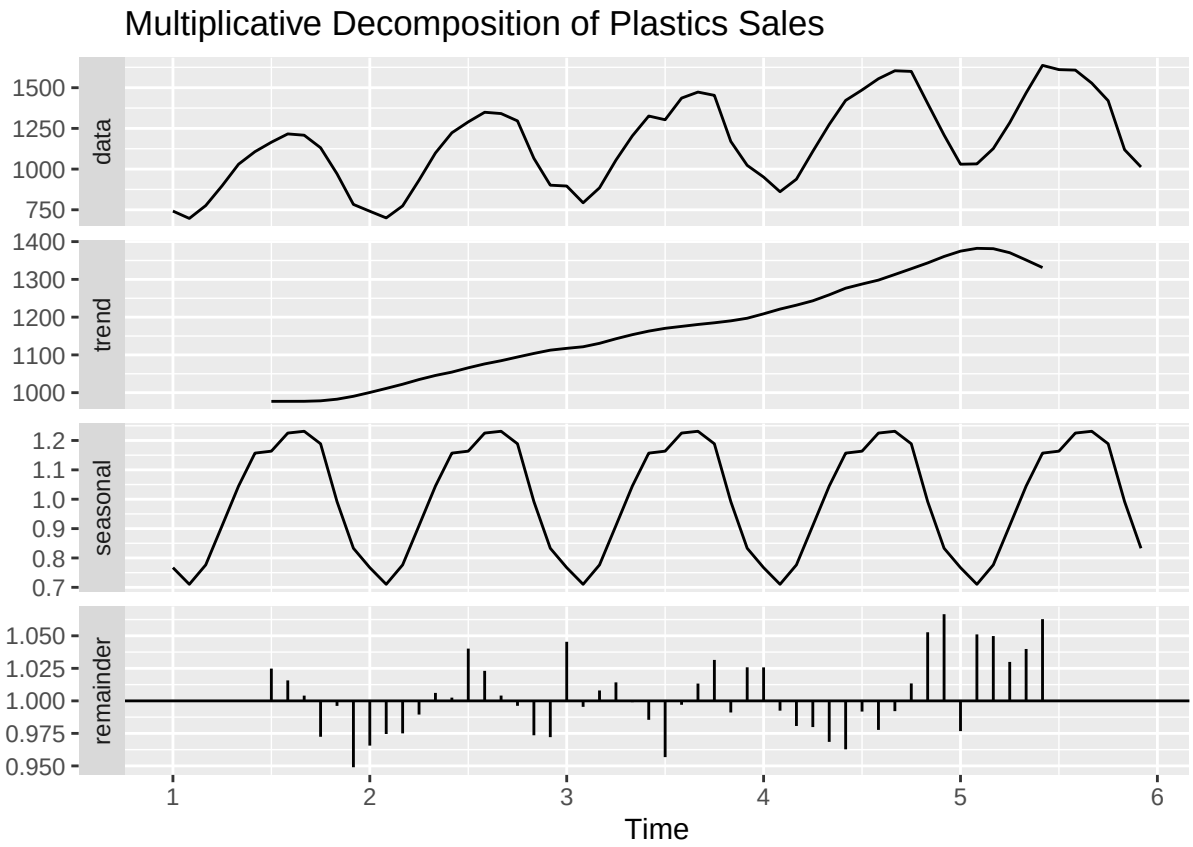
```
autoplot(plastics, main = "Monthly Sales of Product A (Plastics)") +
  ylab("Sales (thousands)") + xlab("Year")
```



After plotting the data, it is clear that there is a seasonal component.

b)

```
decomp_mult <- decompose(plastics, type = "multiplicative")
autoplot(decomp_mult, main = "Multiplicative Decomposition of Plastics Sales")
```




```
# Extract trend-cycle and seasonal indices
trend_cycle <- decomp_mult$trend
seasonal_indices <- decomp_mult$seasonal

# Display seasonal indices (for one year cycle)
print(unique(seasonal_indices)[1:12])
```

```
##  [1] 0.7670466 0.7103357 0.7765294 0.9103112 1.0447386 1.1570026 1.1636317
##  [8] 1.2252952 1.2313635 1.1887444 0.9919176 0.8330834
```

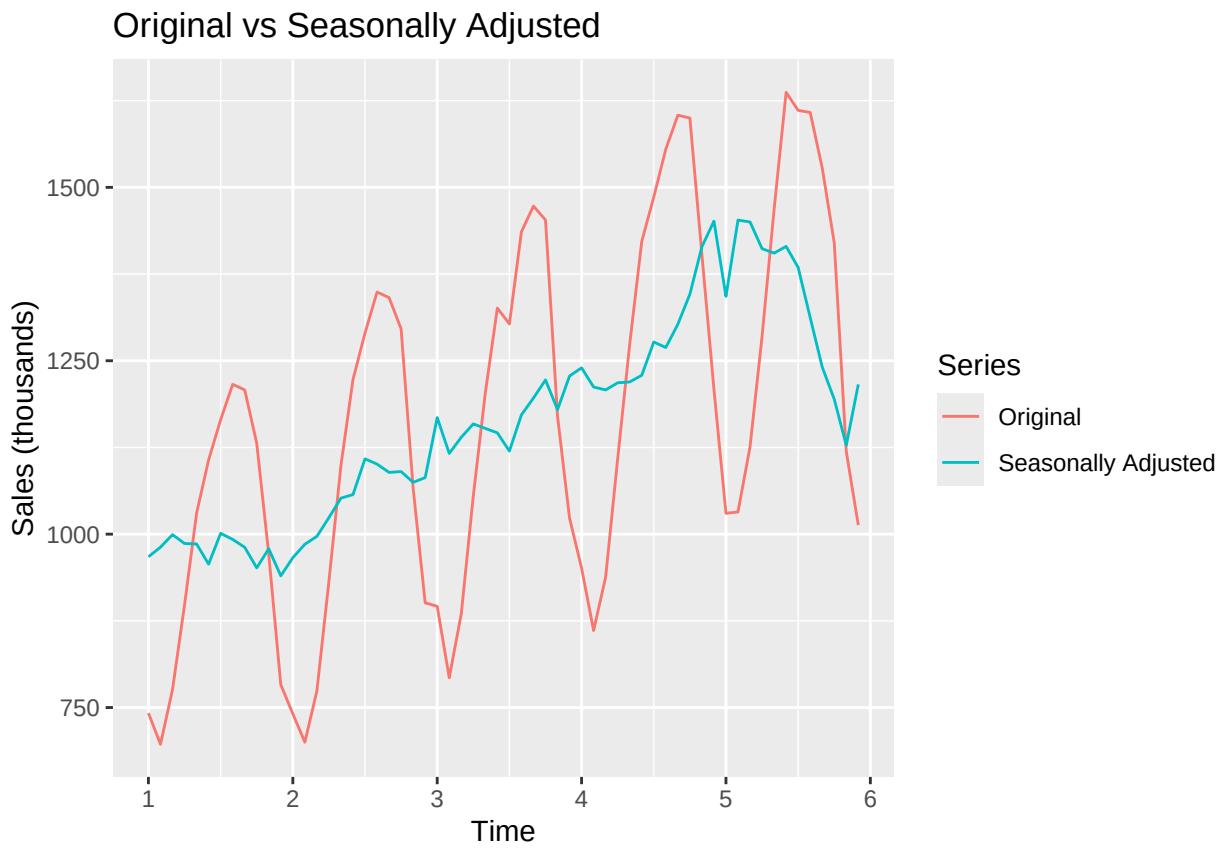
c)

Yes, now that it has been decomposed, it is very clear that the seasonal component is relevant as it fluctuates between .7 and 1.2 which is a lot for multiplicative decomposition.

d)

```
seas_adj <- plastics / decomp_mult$seasonal

autoplot(plastics, series = "Original", main = "Original vs Seasonally Adjusted") +
  autolayer(seas_adj, series = "Seasonally Adjusted") +
  ylab("Sales (thousands)") +
  guides(colour = guide_legend(title = "Series"))
```

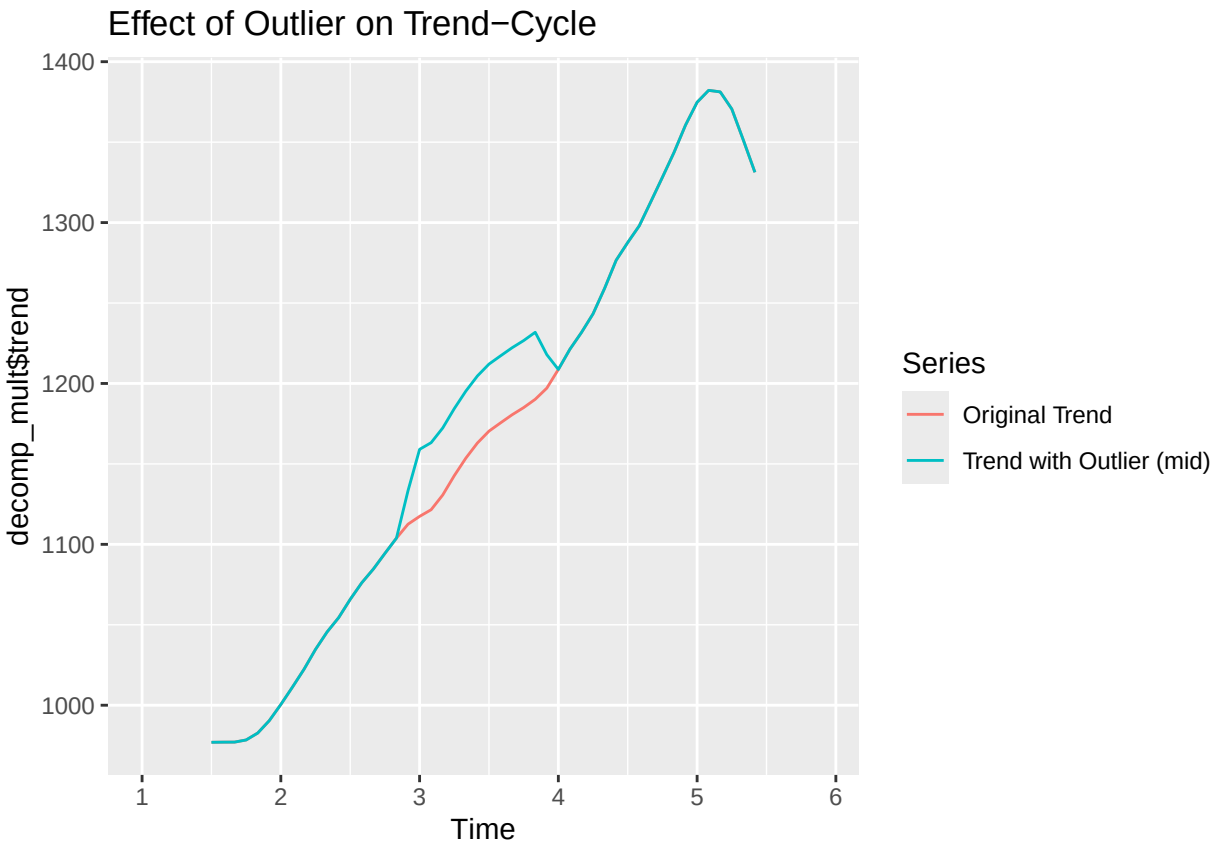


e)

```
plastics_outlier_mid <- plastics
plastics_outlier_mid[30] <- plastics_outlier_mid[30] + 500

decomp_outlier_mid <- decompose(plastics_outlier_mid, type = "multiplicative")
seas_adj_outlier_mid <- plastics_outlier_mid / decomp_outlier_mid$seasonal


autoplot(decomp_mult$trend, series = "Original Trend") +
  autolayer(decomp_outlier_mid$trend, series = "Trend with Outlier (mid)") +
  ggtitle("Effect of Outlier on Trend-Cycle") +
  guides(colour = guide_legend(title = "Series"))
```



Yes, the outlier does make a difference as it distorts the seasonally adjusted trend.

f)

After changing the observation with the outlier to observation 50, there is slightly less distortion in the data, however, it still has an effect on some of the surrounding observations. By changing one in the middle it has an effect on a slightly larger window, but not too much of a difference was noticed.

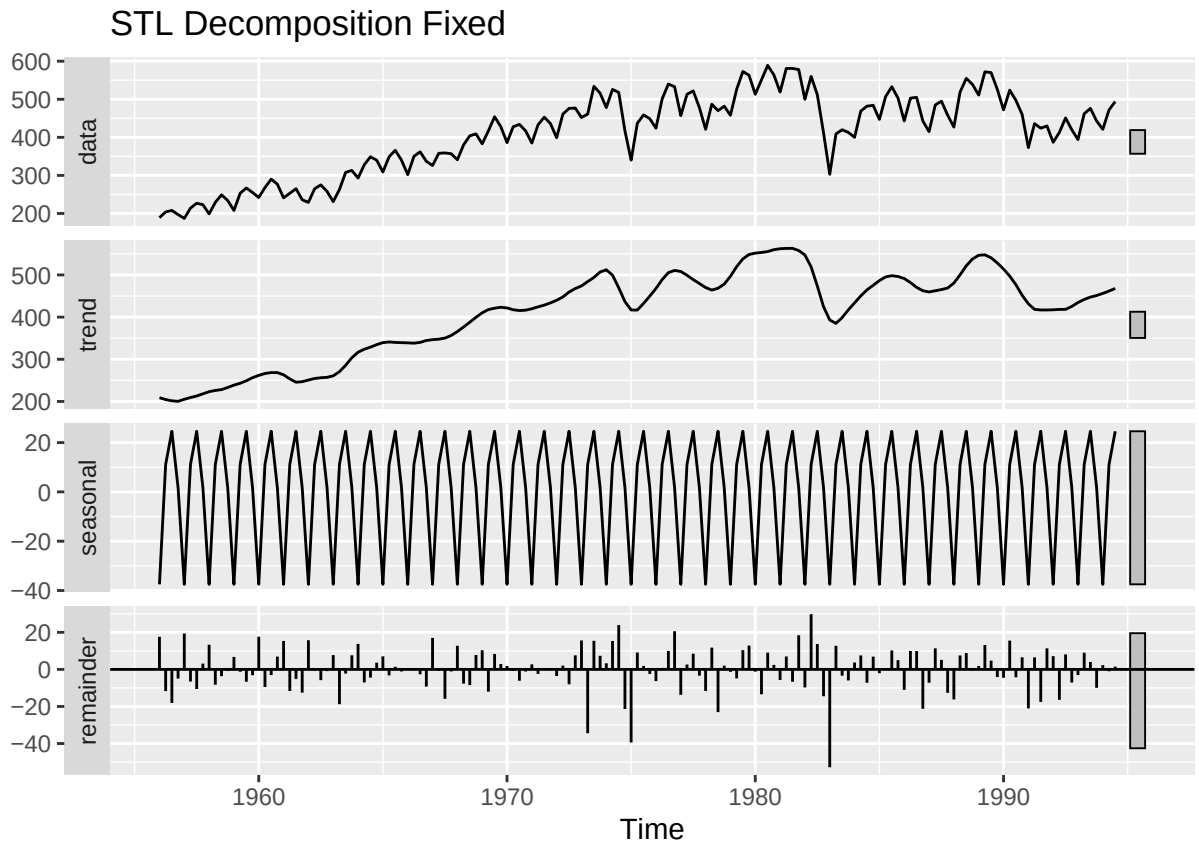
Problem 8

a)

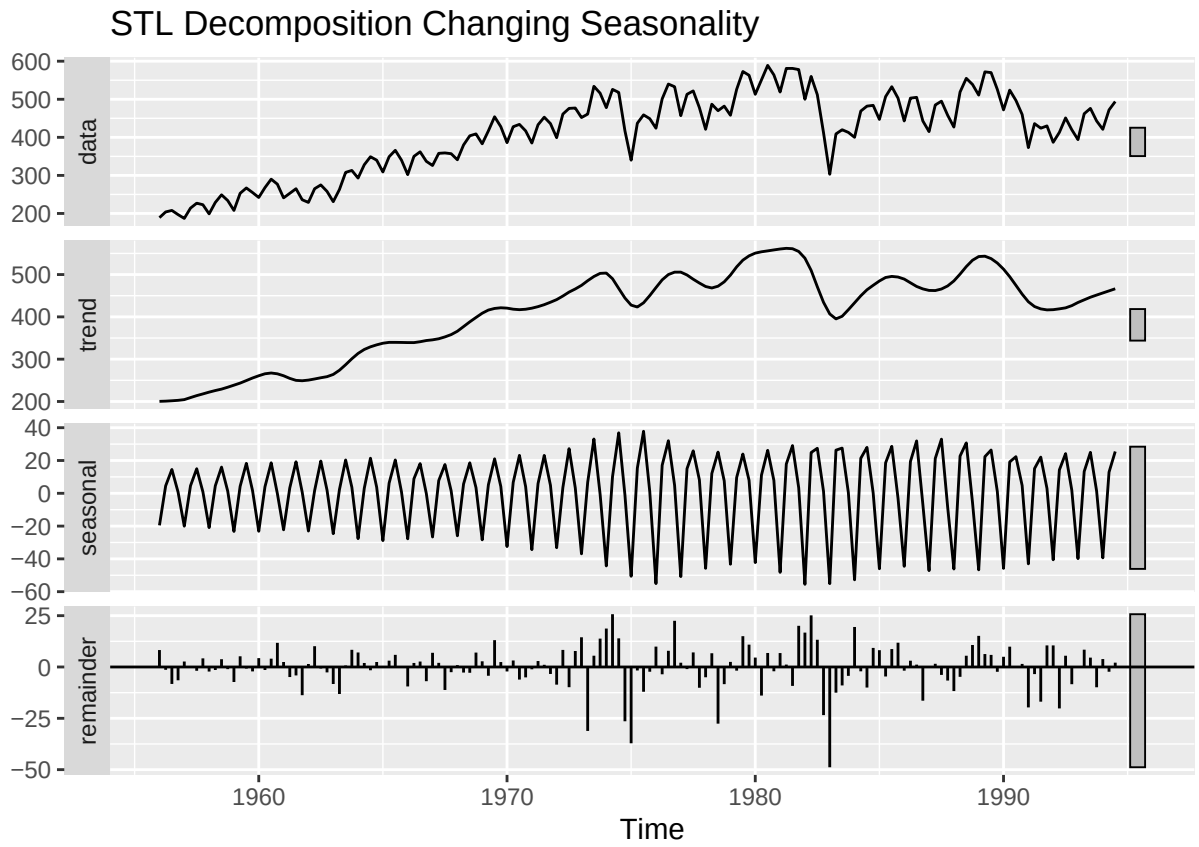
```
data(bricksq)
head(bricksq)
```

```
##      Qtr1 Qtr2 Qtr3 Qtr4
## 1956  189  204  208  197
## 1957  187  214
```

```
stl_fixed <- stl(bricksq, s.window = "periodic")
seasonal_fixed <- stl_fixed$time.series[, "seasonal"]
autoplot(stl_fixed, main = "STL Decomposition Fixed")
```

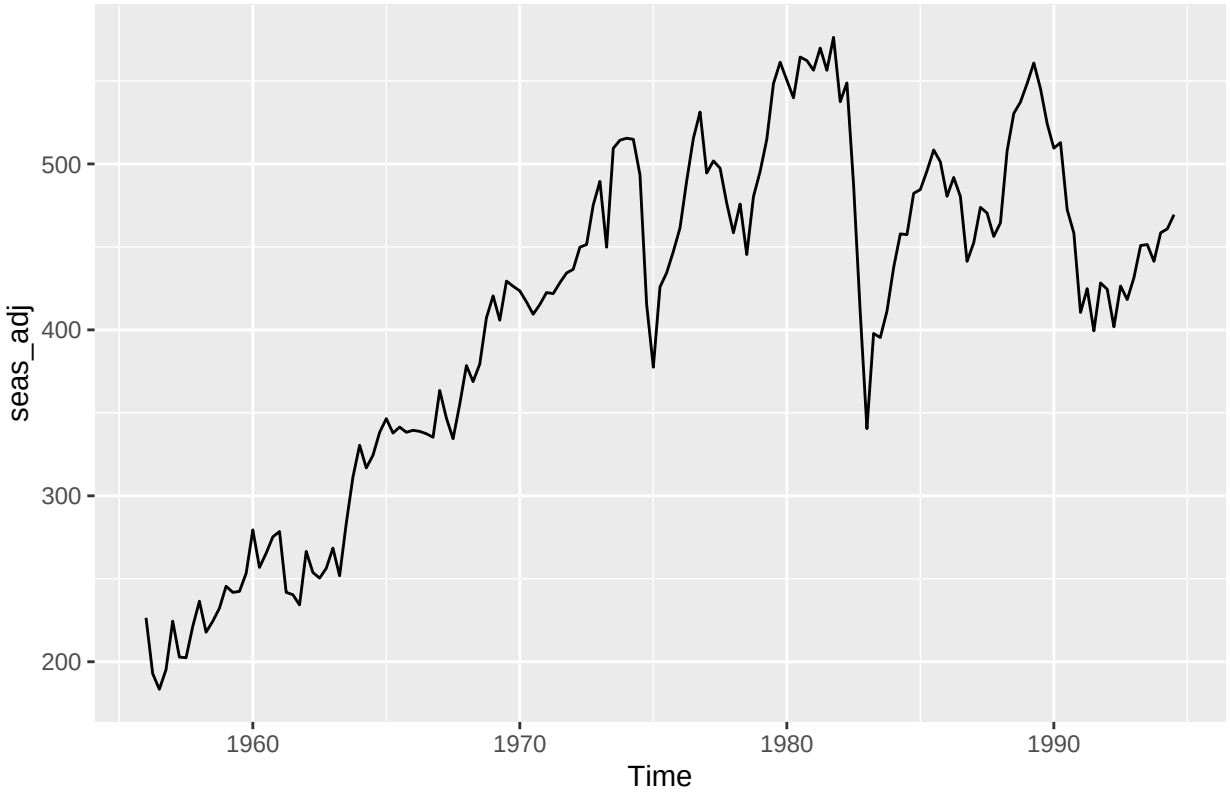



```
stl_changing <- stl(bricksq, s.window = 7)
autoplot(stl_changing, main = "STL Decomposition Changing Seasonality")
```



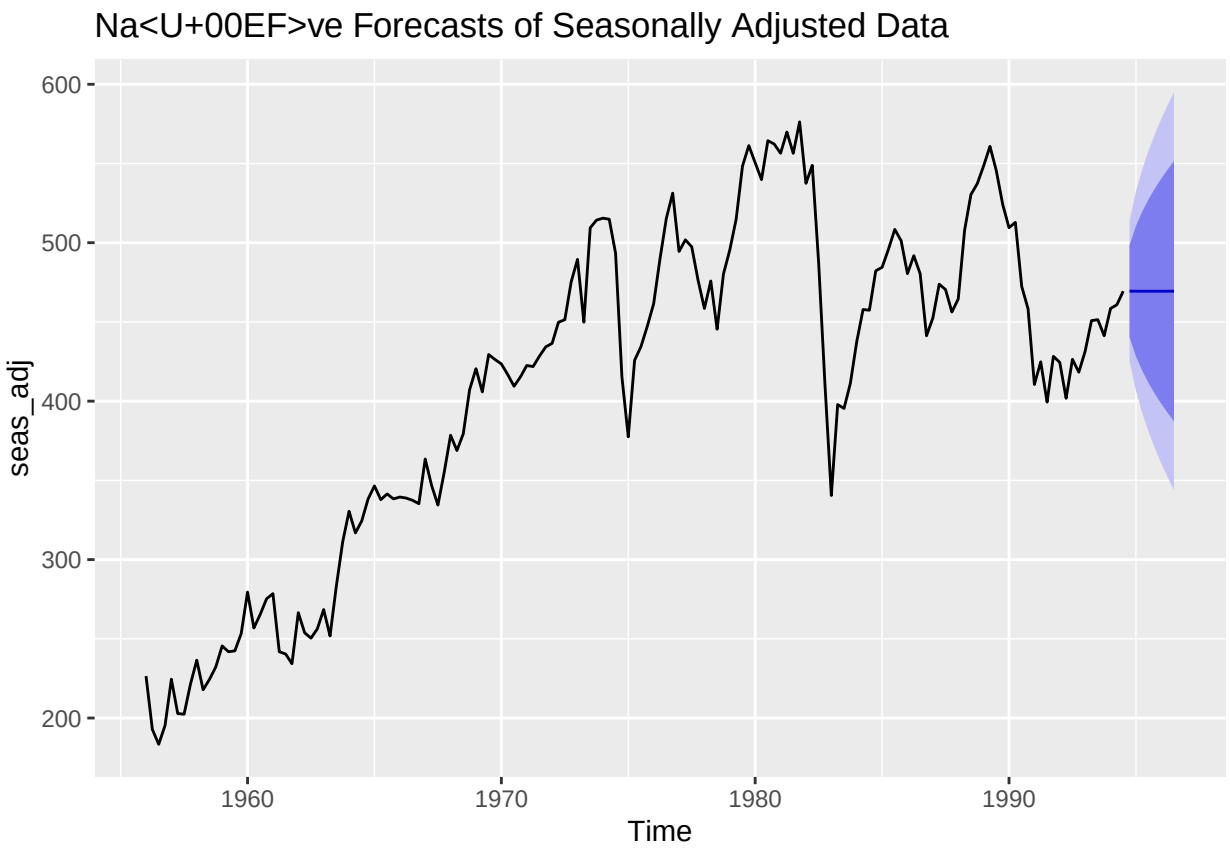
b)

```
seas_adj <- bricksq - seasonal_fixed
autoplot(seas_adj)
```



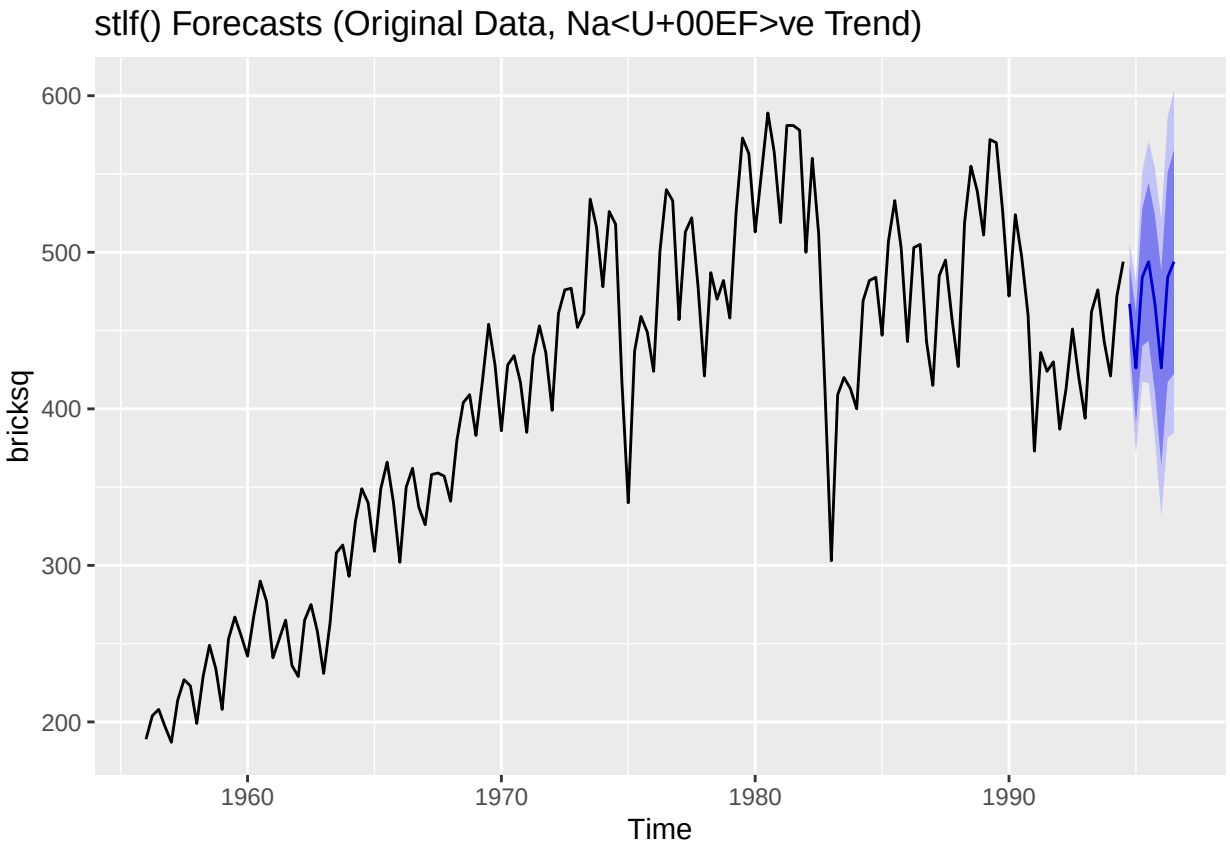
c)

```
naive_sa <- naive(seas_adj, h = 8)
autoplot(naive_sa, main = "Naïve Forecasts of Seasonally Adjusted Data")
```



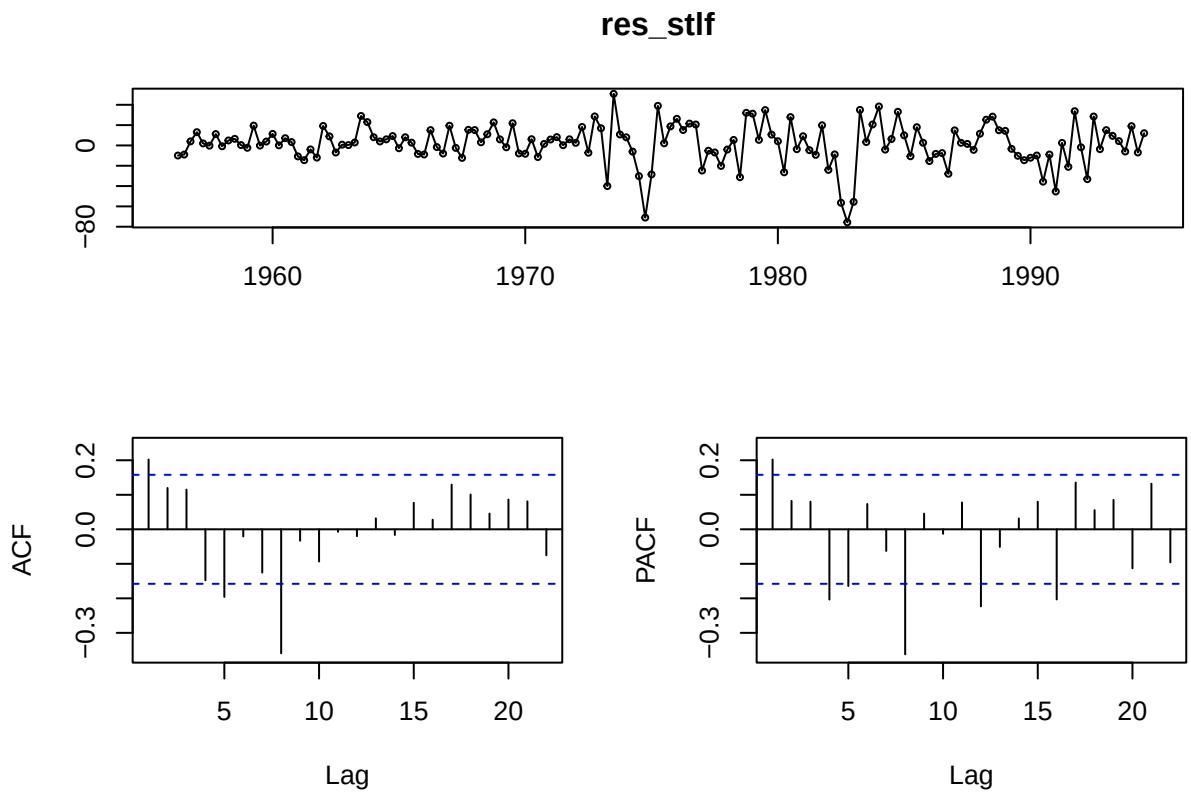
d)

```
stlf_forecast <- stlf(bricksq, h = 8, method = "naive")  # method="naive" for trend
autoplot(stlf_forecast, main = "stlf() Forecasts (Original Data, Naïve Trend)")
```



e)

```
res_stlf <- residuals(stlf(bricksq, h = 8, method = "naive"))
tsdisplay(res_stlf)
```




```
Box.test(res_stlf, lag = 8, type = "Ljung-Box")
```

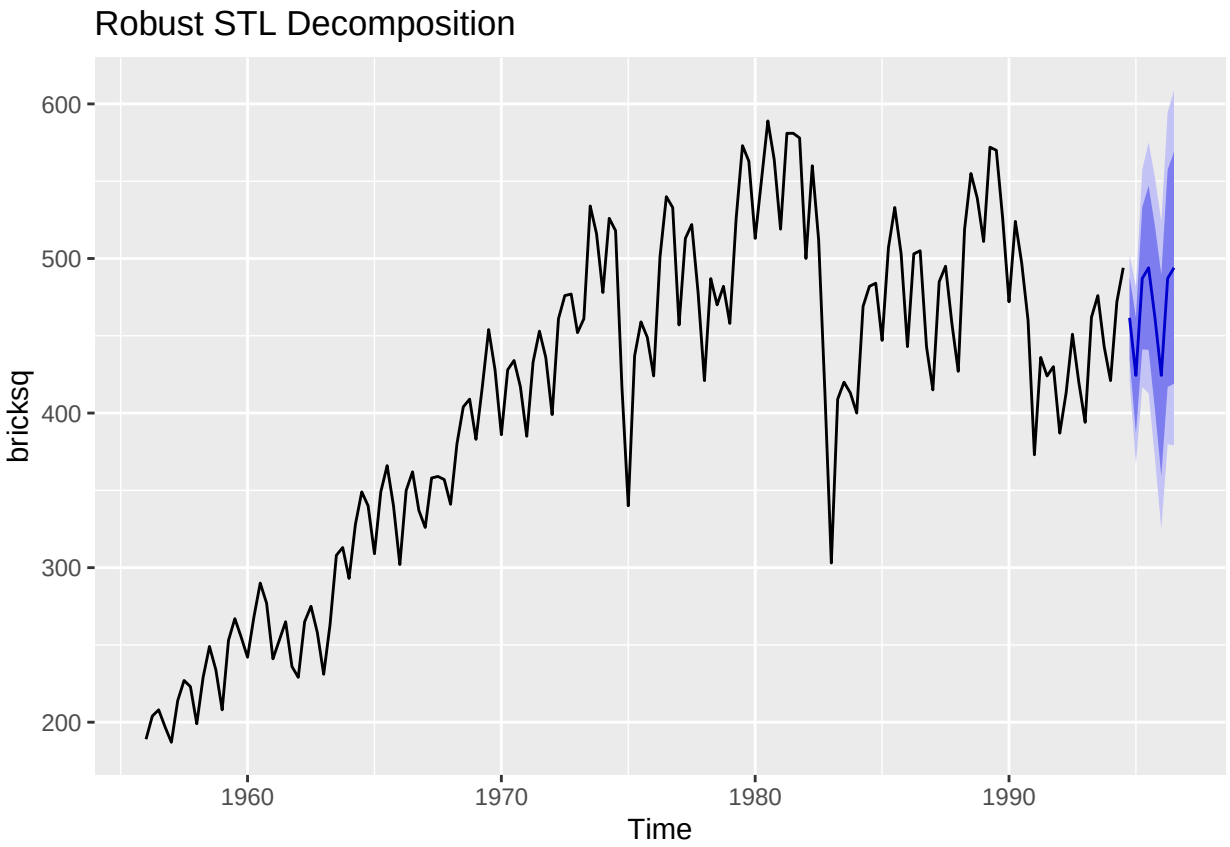
```
##
##  Box-Ljung test
##
## data:  res_stlf
## X-squared = 44.369, df = 8, p-value = 4.845e-07
```

The residuals are not uncorrelated, the ACF and PACF show that there are multiple significant spikes for both while the Box test's significant p-value is strong evidence of autocorrelation in the residuals.

f)

```
robust_stlf_forecast <- stlf(bricksq, h = 8, method = "naive", robust=TRUE)

autoplot(robust_stlf_forecast, main = "Robust STL Decomposition")
```




```
res_stlf_robust <- residuals(robust_stlf_forecast)
tsdisplay(res_stlf_robust)
```




```
Box.test(res_stlf_robust, lag = 8, type = "Ljung-Box")
```

```
##
##  Box-Ljung test
##
## data:  res_stlf_robust
## X-squared = 27.073, df = 8, p-value = 0.0006869
```

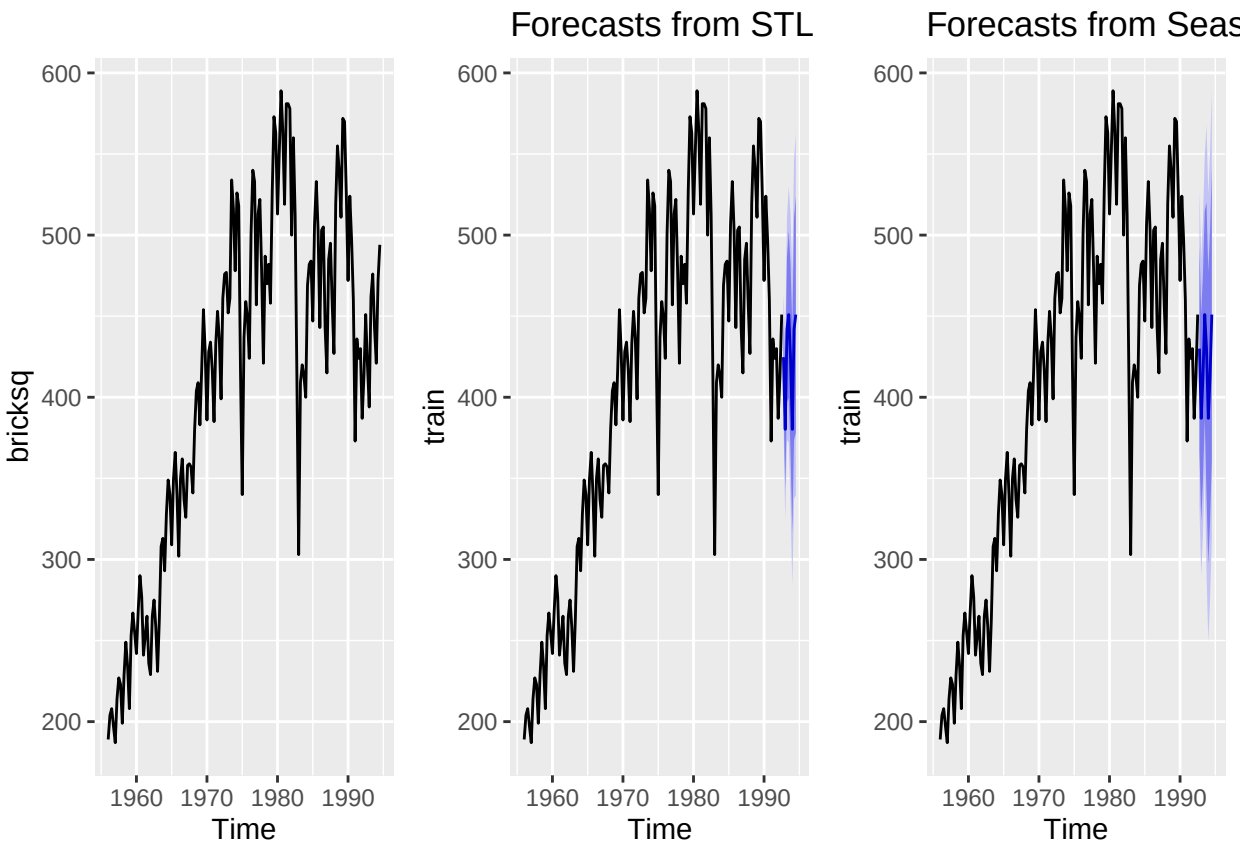
The robust did make a difference and the p value increased a little bit, however, autocorrelation is still present at a significant level from ACF, PACF, and Box-Ljung test.

g)

```
library(gridExtra)
n <- length(bricksq)
h <- 8
train <- window(bricksq, end = time(bricksq)[n - h])
test <- window(bricksq, start = time(bricksq)[n - h + 1])

# Forecasts
fc_stlf <- stlf(train, h = h, method = "naive")
fc_snaive <- snaive(train, h = h)
```

```
plot1=autoplot(bricksq)
plot2=autoplot(fc_stlf)
plot3=autoplot(fc_snaive)
grid.arrange(plot1, plot2, plot3, ncol = 3)
```




```
accuracy(fc_stlf, test)
```

```
##                     ME     RMSE      MAE       MPE     MAPE      MASE      ACF1
## Training set  1.466464 20.12187 14.60088 0.3619027 3.588877 0.4009845 0.2146438
## Test set     23.287082 27.37105 24.47130 5.1081345 5.390090 0.6720560 0.2710260
##              Theil's U
## Training set        NA
## Test set     0.7147682
```

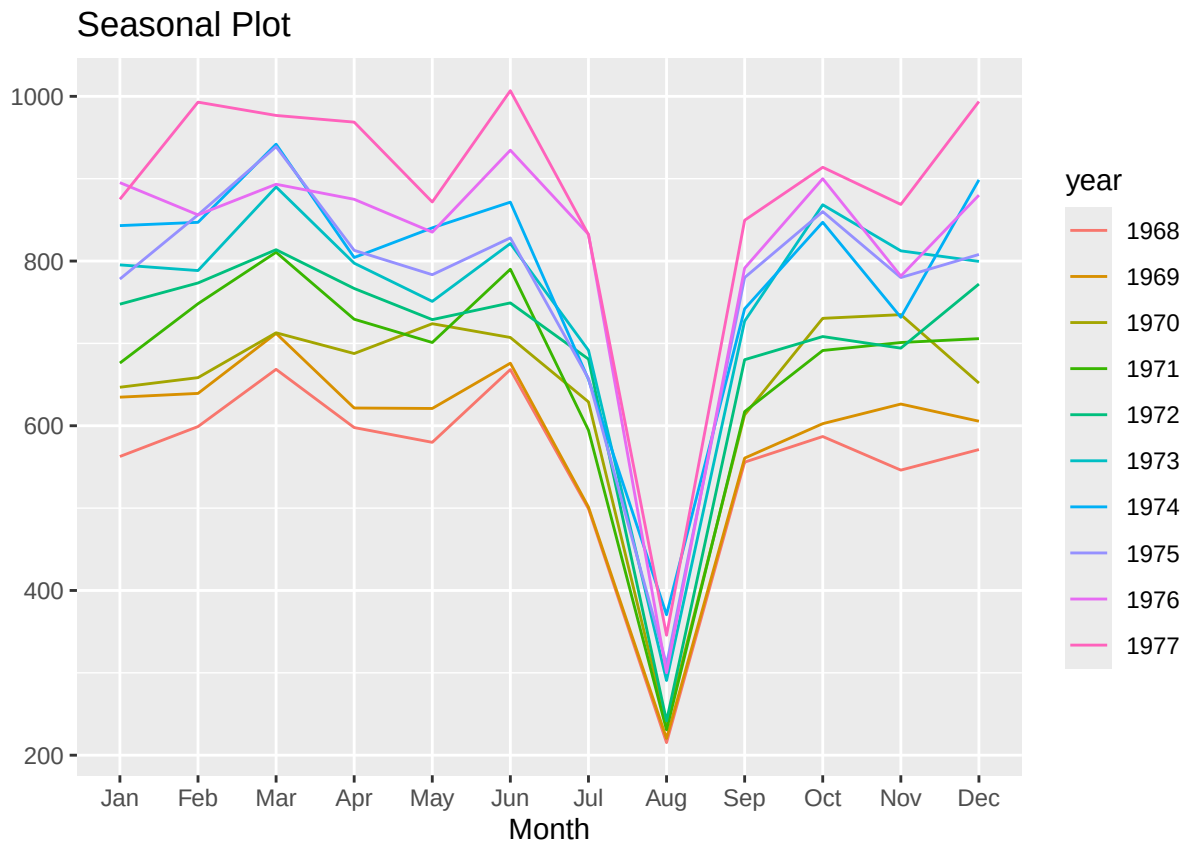
```
accuracy(fc_snaive, test)
```

```
##                     ME     RMSE      MAE      MPE     MAPE      MASE      ACF1
## Training set  6.174825 49.71281 36.41259 1.369661 8.903098 1.0000000 0.8105927
## Test set     27.500000 35.05353 30.00000 5.933607 6.528845 0.8238909 0.2405423
##              Theil's U
## Training set        NA
## Test set     0.9527794
```

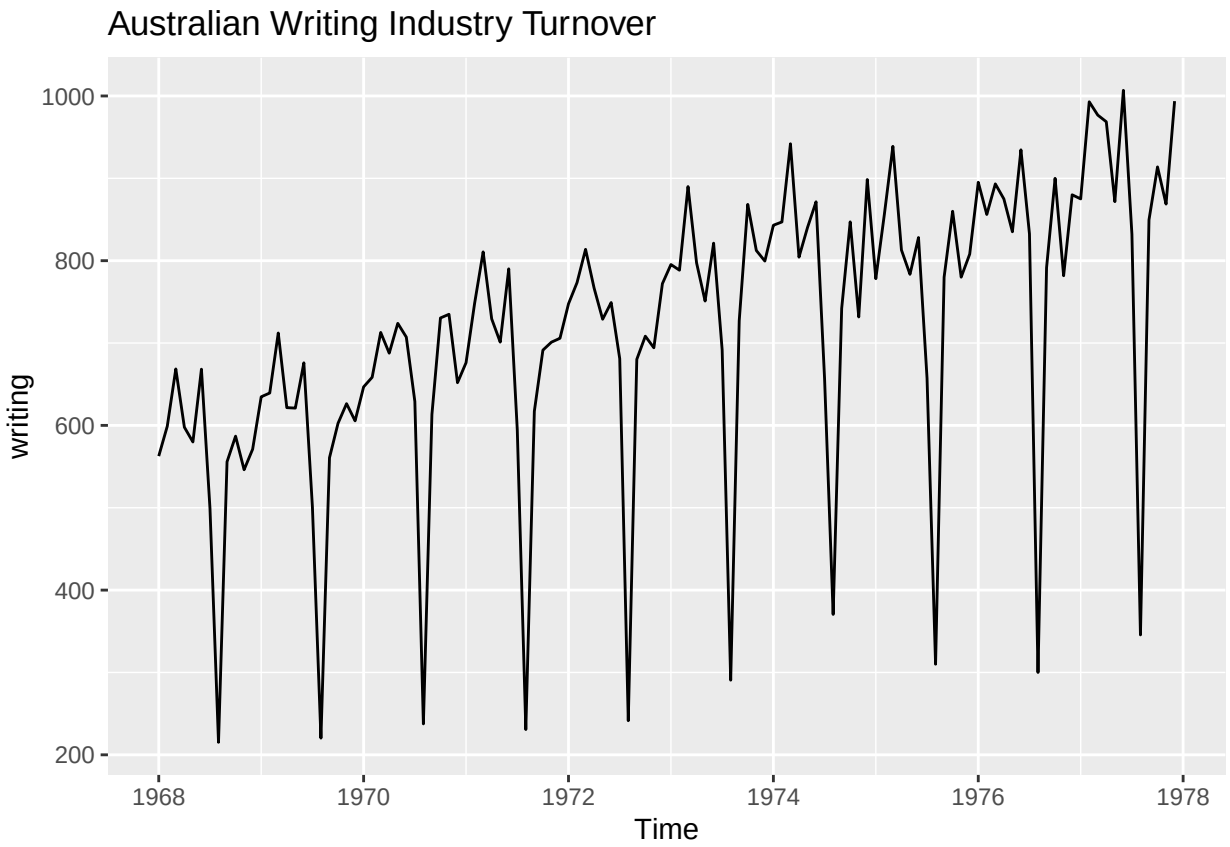
From the accuracy metrics such as ME and RMSE, the STLF was more accurate on the test data compared to the seasonal naive model. This is likely because the trend component being present means that STLF is able to model this trend component and include it in the model while seasonal naive is not able to do that.

Problem 9

```
data(writing)
ggseasonplot(writing, main = "Seasonal Plot")
```




```
autoplot(writing, main = "Australian Writing Industry Turnover")
```

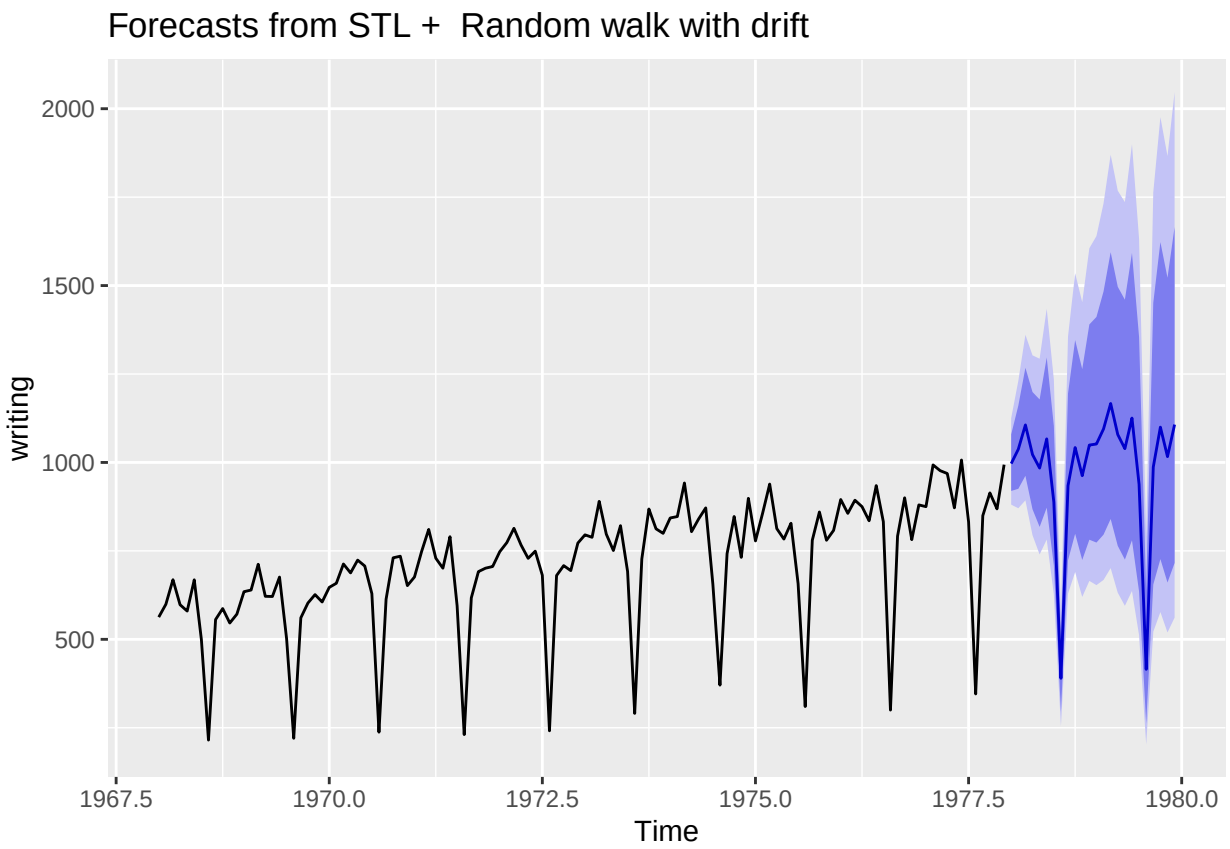



```
BoxCox.lambda(writing)
```

```
## [1] 0.1557392
```

```
fc_rwdrift <- stlf(writing, h = 24, method = "rwdrift", lambda=.1557)

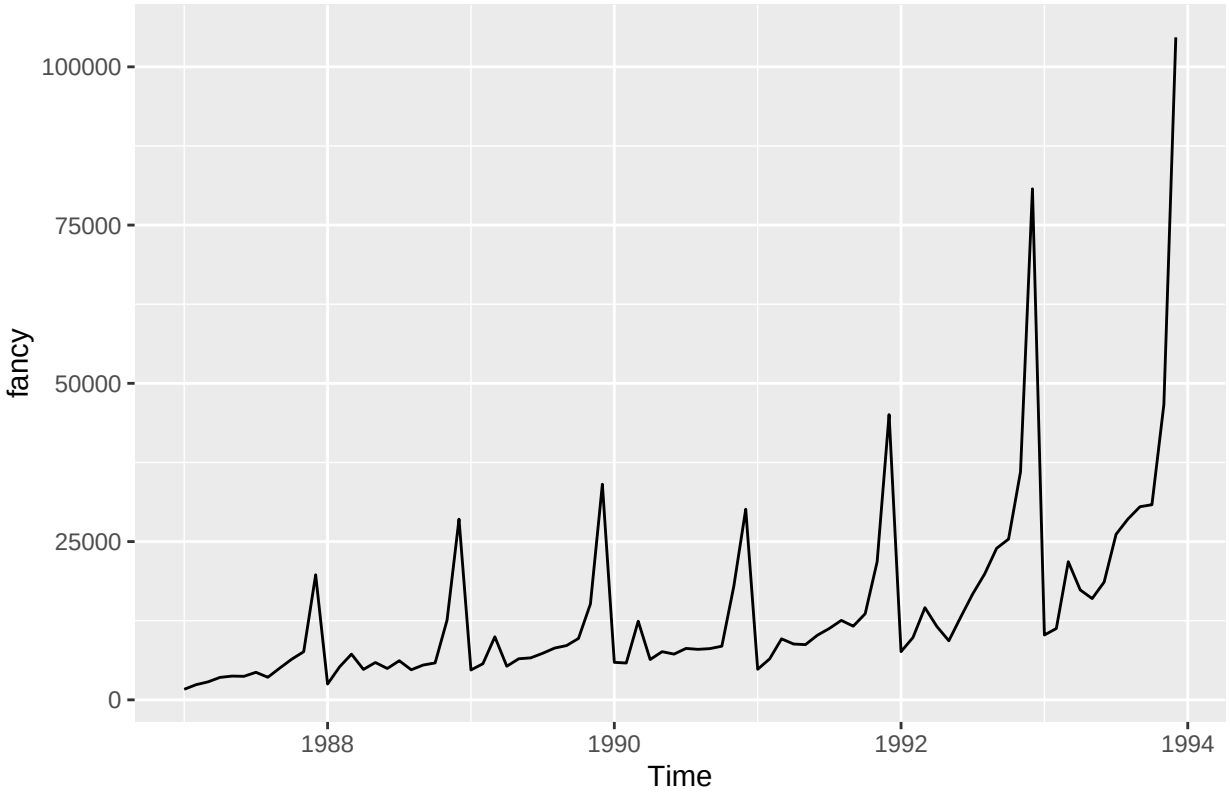
autoplot(fc_rwdrift)
```



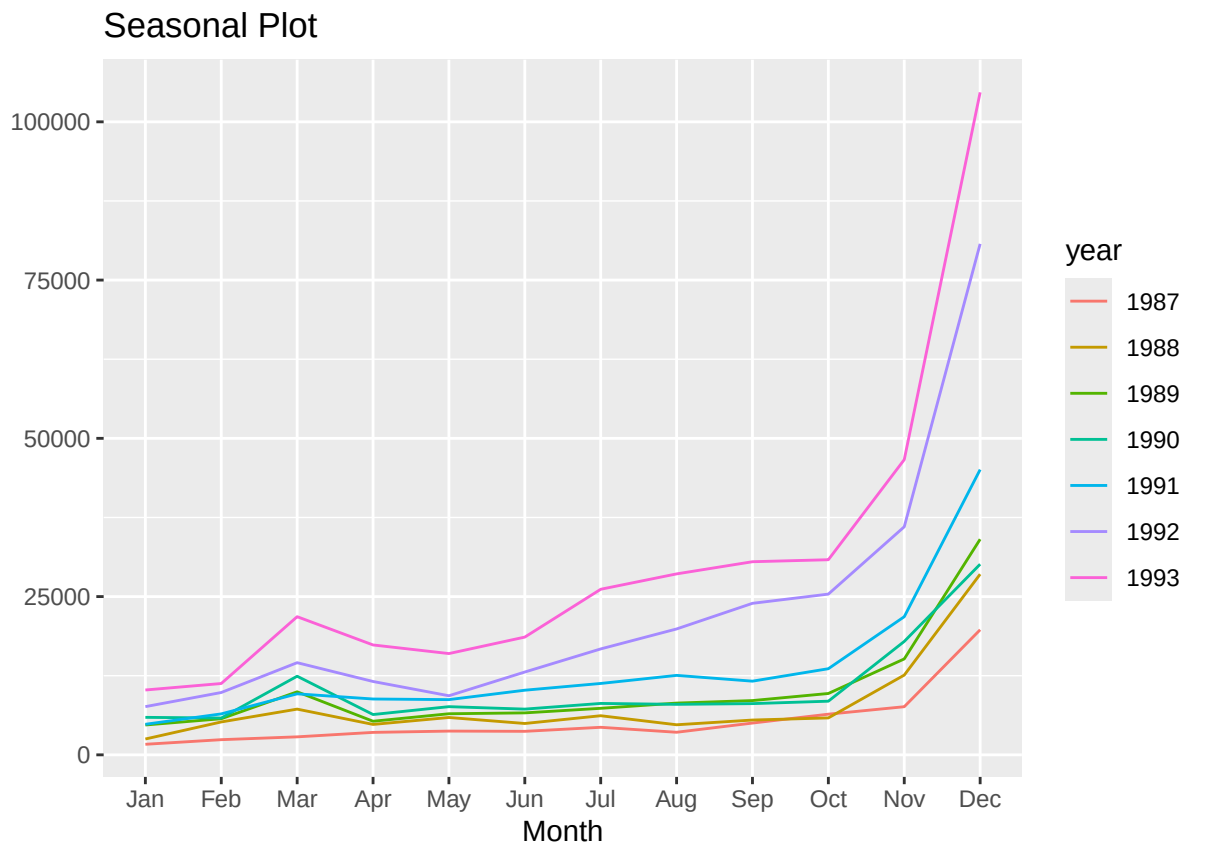
Here it is clear that method rwdrift is preferred due to presence of a trend so I used that method. Also, because the optimal box-cox lambda is .1557, it should also be used to transform the data

Problem 10

```
data(fancy)
autoplot(fancy)
```




```
ggseasonplot(fancy, main = "Seasonal Plot")
```

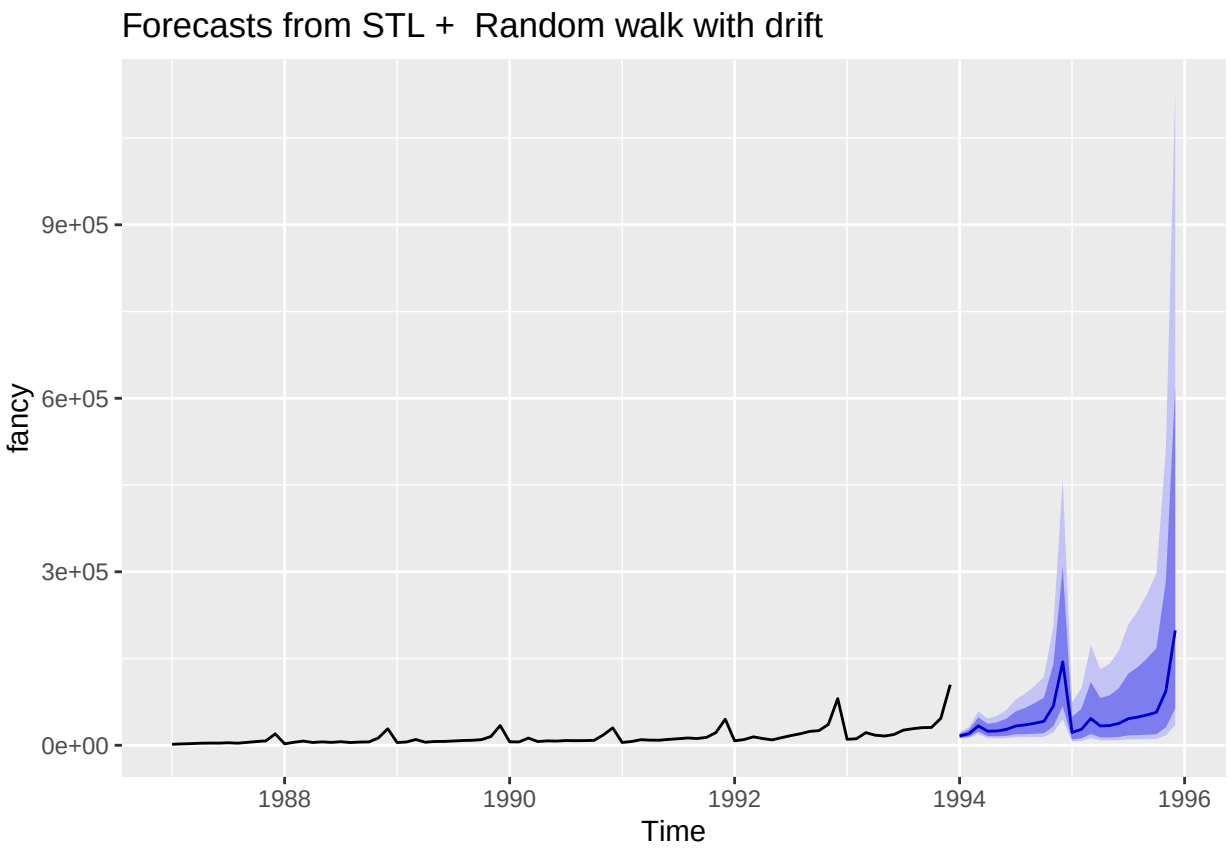



```
BoxCox.lambda(fancy)
```

```
## [1] 0.002127317
```

```
fancy_rwdrift <- stlf(fancy, h = 24, method = "rwdrift", lambda=.002)

autoplot(fancy_rwdrift)
```



Here, a box-cox transformation is very relevant. This is seen through the increasing variance in the seasonality and is backed up by a very low optimal box-cox value. There is also a trend present which indicates that rwdrift is preferred to naive.